Claude Code w praktyce
Zadania

Pracujesz na danych fikcyjnej agencji marketingowej DEMO-STUDIO.

Zasady pracy

1. Agenta uruchamiasz zawsze w folderze **demo-studio** (polecenie `claude`).
2. Przed każdym zadaniem robisz punkt kontrolny w Git — polecenie jest zawsze w kroku 1.
3. Wynik sprawdzasz **sam**, poleceniem w terminalu. Nie pytasz agenta, czy się udało.
4. Gdy agent prosi o zgodę — przeczytaj, co chce zrobić, zanim zatwierdzisz.
5. Windows: polecenia wpisujesz w **Git Bash**, nie w PowerShell.

Zadania na dziś

Nr	Zadanie	Czas	Folder z plikami
1	Przygotowanie	15 min	—
2	Plik CLAUDE.md	30 min	pliki-do-zadan/zadanie-2
3	Komenda /raport	25 min	pliki-do-zadan/zadanie-3
4	Porządek w briefach	40 min	pliki-do-zadan/zadanie-4
5	Git: cofanie zmian	20 min	—
6	Subagenci	30 min	pliki-do-zadan/zadanie-6
7	Mój agent (domowe)	na 13.11	pliki-do-zadan/zadanie-7

Co jest w paczce

- **demo-studio/** — projekt, na którym pracujesz cały dzień
  - **briefy/** — 25 plików od klientów (tylko do odczytu)
  - **zestawienia/** — tu agent zapisuje wyniki
- **pliki-do-zadan/** — pliki potrzebne w zadaniach 2, 3, 4, 6 i 7
- **INSTRUKCJE.pdf** — ten plik

Wszystkie firmy, osoby i adresy e-mail są fikcyjne.

ZADANIE 1

Przygotowanie

⏱ 15 min

> **Cel:** każdy ma działający projekt z punktem kontrolnym w Git.

Git pyta „Please tell me who you are”? Wpisz raz:
`git config --global user.name "Imię Nazwisko"`
`git config --global user.email "twoj@email.pl"`
i powtórz commit.

Krok po kroku

1. Rozpakuj paczkę do folderu **Dokumenty/szkolenie** (ścieżka bez spacji i polskich znaków).
2. Otwórz terminal (Windows: Git Bash) i wejdź do projektu:

```
cd Dokumenty/szkolenie/dzien2-uczestnik/demo-studio
```

3. Sprawdź, czy masz narzędzia:

```
git --version
claude --version
```

✔ **Wynik:** dwa numery wersji.

4. Załóż repozytorium i pierwszy punkt kontrolny:

```
git init
git add .
git commit -m "start"
```

5. Policz briefy:

```
ls briefy | wc -l
```

✔ **Wynik:** 25

6. Uruchom agenta:

```
claude
```

WKLEJ AGENTOWI
Opisz w 3 zdaniach, co jest w tym projekcie. Niczego nie zmieniaj.

Zakończ sesję:

```
/exit
```

Gotowe, gdy:
☐ `git log --oneline` pokazuje commit „start”
☐ w folderze briefy/ jest 25 plików
☐ agent opisał projekt i niczego nie zmienił

ZADANIE 2

Plik CLAUDE.md

⏱ 30 min

> **Cel:** agent przestrzega zasad firmy, chociaż nie powtarzasz ich w poleceniu.

Pliki: pliki-do-zadan/zadanie-2/**wymagania.txt**, **testy.txt**

Jeśli agent prosi o zgodę na zmianę albo usunięcie czegoś w folderze briefy/ — **odmów**.

Krok po kroku

1. Zrób punkt kontrolny w Git:

```
git add . && git commit -m "przed zadaniem 2"
```

2. Otwórz **wymagania.txt** — to 8 zasad szefowej agencji.
3. W folderze demo-studio utwórz plik **CLAUDE.md** z tymi zasadami (najwyżej 30 linii). Możesz napisać go sam albo poprosić agenta:

   WKLEJ AGENTOWI
   Utwórz plik CLAUDE.md z zasadami pracy w tym projekcie. Zasady: [tu wklej 8 zasad z wymagania.txt]

4. Otwórz **testy.txt** — jest tam 5 testów. Każdy test robisz w nowej sesji:

```
/exit
claude
```

   i wklejasz polecenie testu dokładnie tak, jak jest.
5. Porównaj zachowanie agenta z „Dobrym wynikiem” w testy.txt i wpisz TAK albo NIE.
6. Test nie wyszedł? Popraw **CLAUDE.md** (nie polecenie!) i powtórz ten test w nowej sesji.

Gotowe, gdy:

☐ plik CLAUDE.md ma najwyżej 30 linii — sprawdź: `wc -l CLAUDE.md`
☐ co najmniej 4 z 5 testów: TAK

Dla chętnych

Sprawdź, co się dzieje przy sprzecznych zasadach. Skopiuj plik ze sprzecznymi regułami do folderu briefy:

```
cp ../pliki-do-zadan/zadanie-2/dla-chetnych-briefy-CLAUDE.md briefy/CLAUDE.md
```

Zrób test 1 dwa razy (dwie nowe sesje). Których zasad posłuchał agent? Na koniec usuń ten plik:

```
rm briefy/CLAUDE.md
```

ZADANIE 3

Komenda /raport

⏱ 25 min

> **Cel:** jedna komenda robi miesięczny raport, a przy błędnym miesiącu niczego nie tworzy.

Pliki: pliki-do-zadan/zadanie-3/**raport.md** (szablon komendy)

Krok po kroku

1. Zrób punkt kontrolny w Git:

```
git add . && git commit -m "przed zadaniem 3"
```

2. Skopiuj szablon komendy do projektu:

```
mkdir -p .claude/commands
cp ../pliki-do-zadan/zadanie-3/raport.md .claude/commands/raport.md
```

3. Otwórz plik **.claude/commands/raport.md** i uzupełnij 4 miejsca oznaczone **UZUPEŁNIJ**. Folder .claude jest ukryty — na Macu pokażesz go skrótem Cmd+Shift+kropka.

4. Uruchom nową sesję i wpisz sam ukośnik:

```
claude
/
```

✔ **Wynik:** na liście jest komenda /raport z Twoim opisem.

5. Przetestuj trzy wywołania. Po każdym sprawdź folder (wykrzyknik = polecenie terminala bez wychodzenia z sesji):

```
!ls zestawienia
```

```
/raport luty
```

✔ **Wynik:** powstaje plik raport-luty.md

```
/raport lutyy
```

✔ **Wynik:** komunikat o błędzie, żaden nowy plik

```
/raport
```

✔ **Wynik:** komunikat o błędzie, żaden nowy plik

6. Policz sam briefy z lutego i porównaj z sekcją „Kontrola kompletności" w raport-luty.md:

```
grep -lE "Termin: (2026-02|[0-9]+\.02\.2026|[0-9]+ lutego)" briefy/* | wc -l
```

Gotowe, gdy:

- ☐ `/raport luty` tworzy plik z tabelą
- ☐ błędny i pusty argument nie tworzą żadnego pliku
- ☐ liczba briefów w raporcie = liczba z polecenia grep

Dla chętnych

Zrób drugą komendę `/porownaj luty marzec` — tabela różnic w liczbie briefów i budżetach według kategorii. Uwaga: w komendach `$0` to pierwszy argument, a `$1` to drugi.

ZADANIE 4

Porządek w briefach

⏱ 40 min

> **Cel:** agent porządkuje 25 plików i żadnego nie gubi po cichu.

Pliki: pliki-do-zadan/zadanie-4/**karta-sprawdzenia.txt**

Niektóre pliki tylko udają briefy albo mają niejasne dane. To celowe — agent nie może ich zgubić bez słowa.

Krok po kroku

1. Zrób punkt kontrolny w Git:

```
git add . && git commit -m "przed zadaniem 4"
```

2. Przegląd bez zmian:

WKLEJ AGENTOWI
Przeczytaj wszystkie pliki w briefy/. Zapisz zestawienia/przeglad.md z kolumnami: plik, klient, kategoria, termin, budżet, uwagi. Niczego nie kopiuj i nie zmieniaj nazw.

3. Kopia z nowymi nazwami:

WKLEJ AGENTOWI
Skopiuj briefy do nowego folderu briefy-uporzadkowane/ z nazwami RRRR-MM-DD-klient-kategoria.md (data z pola Termin, małe litery, bez polskich znaków, myślniki zamiast spacji). Plików, których nie da się jednoznacznie nazwać albo które nie są briefami, nie kopiuj — wypisz je z przyczyną w zestawienia/do-wyjasnienia.md. Folder briefy/ zostaw bez zmian.

4. Raport za marzec:

```
/raport marzec
```

5. Wyjdź z sesji (`/exit`) i sprawdź sam. Liczby wpisz do **karta-sprawdzenia.txt**:

```
ls briefy | wc -l
ls briefy-uporzadkowane | wc -l
git status briefy/
```

Otwórz też **zestawienia/do-wyjasnienia.md** i policz pominięte pliki.

6. Skopiowane + pominięte = 25? Jeśli nie — znajdź plik, który zniknął, popraw polecenie i powtórz.
7. W karcie wypisz pliki, które sprawiły problem, i co zrobił z nimi agent.

Gotowe, gdy:

☐ skopiowane + pominięte = 25
☐ `git status briefy/` nie pokazuje zmian
☐ karta sprawdzenia wypełniona

Dla chętnych
Dopisz do komendy /raport zasady dla duplikatów, dat bez roku i dat niejednoznacznych. Usuń raport-marzec.md i uruchom `/raport marzec` jeszcze raz.

ZADANIE 5

Git: cofanie zmian

⏱ 20 min

> **Cel:** umiesz cofnąć zmiany agenta i wiesz, czego samo „cofnij" nie usuwa.

Krok po kroku

1. Punkt kontrolny — tym razem obowiązkowo:

```
git add . && git commit -m "przed zadaniem 5"
git status
```

✔ **Wynik:** nothing to commit, working tree clean

2. Uruchom agenta (`claude`) i wklej:

WKLEJ AGENTOWI
To ćwiczenie na kopii danych. Wyjątkowo zezwalam na zmiany w briefy/. Zmień nazwy wszystkich plików w briefy/ na wzór RRRR-MM-DD-klient-kategoria.md i w każdym pliku zapisz nazwę klienta wielkimi literami.

Zatwierdzaj zmiany.

3. Poproś o podsumowanie, potem wyjdź:

WKLEJ AGENTOWI
Na podstawie git status podsumuj, co się zmieniło. Niczego nie przywracaj.

```
/exit
```

4. Cofnij zmiany w plikach:

```
git checkout -- .
```

5. Policz pliki i sprawdź status:

```
ls briefy | wc -l
git status
```

Ile jest plików? Dlaczego więcej niż 25?

6. Najpierw zobacz, co Git usunie (podgląd), potem usuń nowe pliki:

```
git clean -n     # podgląd - nic nie kasuje
git clean -fd    # usuwa nowe pliki
```

7. Sprawdź:

```
git status
ls briefy | wc -l
```

✔ **Wynik:** nothing to commit, working tree clean oraz 25

Gotowe, gdy:

- ☐ `git status` jest czysty
- ☐ w briefy/ jest 25 plików
- ☐ umiesz w jednym zdaniu powiedzieć, dlaczego krok 4 nie wystarczył

Dla chętnych

Poproś agenta o zmianę treści jednego pliku i cofnij ją poleceniem `/rewind`. Potem poproś o zmianę nazwy pliku i spróbuj cofnąć tak samo. Co zadziałało, a co nie?

ZADANIE 6

Subagenci

⏱ 30 min

> **Cel:** dwaj pomocnicy sprawdzają zestawienie, nie zmieniając go.

Pliki: pliki-do-zadan/zadanie-6/**zestawienie-do-kontroli.md**, **korektor.md** (gotowy wzór)

Krok po kroku

1. Skopiuj plik do kontroli i zrób punkt kontrolny:

```
cp ../pliki-do-zadan/zadanie-6/zestawienie-do-kontroli.md zestawienia/
git add . && git commit -m "przed zadaniem 6"
```

2. Dodaj gotowego subagenta **korektor**:

```
mkdir -p .claude/agents
cp ../pliki-do-zadan/zadanie-6/korektor.md .claude/agents/
```

3. Zrób drugiego subagenta na wzór korektora:

```
cp .claude/agents/korektor.md .claude/agents/analityk-budzetow.md
```

Otwórz nowy plik i zmień: **name** na analityk-budzetow, **description** i instrukcję — ma liczyć liczbę briefów i sumę budżetów w każdej kategorii. Linię `tools: Read, Grep, Glob` zostaw bez zmian. Pierwsza linia pliku musi być `---`.

4. Nowa sesja i sprawdzenie listy:

```
claude
/agents
```

✔ **Wynik:** na liście są korektor i analityk-budzetow.

5. Wklej:

WKLEJ AGENTOWI

Użyj subagentów korektor i analityk-budzetow na pliku zestawienia/zestawienie-do-kontroli.md. Scal ich wyniki w zestawienia/kontrola.md z dwiema sekcjami: Korekta i Budżety. Pliku źródłowego nie zmieniaj.

6. Sprawdź sam, czy plik źródłowy jest nietknięty:

```
git diff zestawienia/zestawienie-do-kontroli.md
```

✔ **Wynik:** pusto (brak zmian).
Policz ręcznie sumę budżetów w kategorii **newsletter** i porównaj z analitykiem.

7. Przeczytaj sekcję Budżety w kontrola.md. Czy coś się nie zgadza? Napisz jedno zdanie na czacie.

Gotowe, gdy:

- ☐ `/agents` pokazuje obu subagentów
- ☐ plik źródłowy niezmieniony (git diff pusty)
- ☐ kontrola.md ma sekcje Korekta i Budżety
- ☐ Twoja odpowiedź z kroku 7 jest na czacie

Dla chętnych

Wypełnij
dla-chetnych-karty-delegowania.txt
: przy 6 zadaniach zdecyduj, czy warto je dzielić między subagentów.

ZADANIE 7

Mój agent — zadanie domowe

⏱ na 13.11 · prezentacja 3 min

> **Cel:** przenosisz dzisiejsze techniki na zadanie z własnej pracy.

Pliki: pliki-do-zadan/zadanie-7/**karta-projektu.txt**

Krok po kroku

1. Wybierz zadanie, które powtarza się co najmniej raz w tygodniu i działa na plikach (raporty, zestawienia, porządkowanie, korekta).
2. Załóż nowy folder projektu z danymi **przykładowymi** — bez prawdziwych danych osobowych i bez kluczy API.
3. Zrób `git init` i commit, zanim pierwszy raz uruchomisz agenta.
4. Napisz CLAUDE.md (do 30 linii) i jedną komendę własną z argumentem. Subagent — jeśli chcesz.
5. Zapisz w karcie 4 kryteria TAK/NIE. Każde musi dać się sprawdzić bez pytania agenta.
   Źle: „raport jest czytelny". Dobrze: „raport ma tyle wierszy, ile plików w folderze wejściowym".
6. Uruchom, sprawdź kryteria, popraw. Uzupełnij kartę projektu.

Gotowe, gdy:

☐ karta projektu wypełniona
☐ komenda działa — pokażesz ją na żywo 13.11 (3 minuty)